Transport Layer

Integrantes:
Lucas do Nascimento Macedo lucasnascimento77@hotmail.com
Miguel Sanches Rocha miguel.sanches98b@gmail.com

Sumário

- 6.1 - Questões de design da camada de transporte
  - 6.1.1 - Serviços prestados à camada de sessão
  - 6.1.2 - Qualidade do serviço
  - 6.1.3 - As primitivas de serviços de transporte OSI
  - 6.1.4 - Protocolos de transporte
  - 6.1.5 - Elementos de protocolos de transporte
- 6.2 - Gerenciamento de conexão
  - 6.2.1 - Endereçamento
  - 6.2.2 - Estabelecendo uma conexão
  - 6.2.3 - Liberando a conexão
  - 6.2.4 - Gerenciamento da conexão baseado em contador

Referências bibliográficas

- Andrew S. Tanenbaum. 1988. Computer networks: 2nd edition. Prentice-Hall, Inc., USA.

6.1 - Questões de design da camada de transporte

- Nesta divisão, são tratadas algumas questões que os profissionais envolvidos com desenvolvimento de camadas de transporte devem se ater.
- Elementos como o tipo de serviço prestado à camada de sessão, a qualidade deste serviço e as primitivas da camada de transporte fornecidas para a realização dos serviços serão discutidos.
- Ao final, será elaborada uma discussão acerca dos protocolos de transporte necessários para que todo o ambiente na camada de transporte e seus serviços, sejam implementados corretamente.

○ 6.1.1 - Serviços prestados à camada de sessão

- A grande tarefa atribuída à camada de transporte é a de fornecer serviços eficientes, confiáveis e com bom custo-benefício, geralmente para entidades da camada imediatamente superior (camada de sessão). Assim, a camada de transporte realiza o transporte e regula o fluxo de informações da origem até o destino, de forma confiável e eficiente.
- Para alcançar tal objetivo, a camada de transporte faz uso de recursos provindos da camada imediatamente inferior (camada de rede).
- A unidade de software e/ou hardware dentro da camada de transporte que está encarregada de realizar este trabalho é chamada de “entidade de transporte”.
- Existem dois tipos de serviços de transporte: Orientado à conexão e sem conexão. São muito parecidos com os serviços da camada de rede.
- Serviço orientado à conexão possui três estágios (estabelecimento, transferência de dados e liberação).

○ 6.1.1 - Serviços prestados à camada de sessão

- Apesar dos serviços da camada de transporte e camada de rede serem bastante similares, é necessário a existência de duas camadas para que a qualidade dos serviços prestados seja consolidada, de modo que não ocorra perdas frequentes de pacotes e dados corrompidos devido a um serviço não confiável.
- Assim, se uma entidade de transporte fica ciente de uma interrupção da conexão de rede em uma longa linha de transmissão, ela consegue administrar uma nova conexão e continuar a transmissão de dados a partir do lugar em que foi interrompida.

○ 6.1.2 - Qualidade do serviço

- Outra forma de observar a camada de transporte é por meio da análise da qualidade do serviço fornecida pela camada de rede. Se o serviço de rede for impecável, a camada de transporte terá facilidade para executar suas funções. Caso o serviço de rede seja ruim, a camada de transporte terá que preencher a lacuna entre o que os usuários de transporte querem e o que a camada de rede está oferecendo.
- A qualidade do serviço pode ser caracterizado por um conjunto de parâmetros. É tarefa da camada de transporte examinar esses parâmetros e determinar se é possível fornecer o serviço requerido levando em conta os serviços de rede disponíveis.
- Os parâmetros e suas definições seguem abaixo:

○ 6.1.2 - Qualidade do serviço

- Atraso de estabelecimento de conexão: Quantidade de tempo decorrido entre a solicitação de uma conexão de transporte e a sua confirmação sendo recebida pelo usuário do serviço de transporte. Quanto menor o atraso, melhor o serviço.
- Probabilidade de falha no estabelecimento da conexão: Chance da conexão não ser estabelecida dentro do tempo máximo de atraso. Pode ocorrer devido a congestionamento na rede, por exemplo.
- *Throughput*: Número de *bytes* transferidos por segundo.
- Atraso de trânsito: Tempo entre o envio da mensagem pelo usuário de transporte e o recebimento no destino.
- Taxa de erro residual: Número de mensagens perdidas ou truncadas como uma fração do total enviado

○ 6.1.2 - Qualidade do serviço

- Probabilidade de falha de transferência: Mede a eficiência do serviço de transporte de acordo com sua função.
- Atraso na liberação da conexão: Quantidade de tempo decorrido entre o início da liberação de uma conexão e a liberação verdadeiramente ocorrida na outra extremidade.
- Probabilidade de falha na liberação da conexão: Fração de tentativas de liberação que não se completaram dentro do intervalo de atraso permitido.
- Protection: Fornece uma forma de especificar o interesse de se ter proteção contra terceiros não autorizados ler ou modificar dados transmitidos.
- Prioridade: Fornece uma forma para o usuário de transporte indicar que algumas de suas conexões são mais importantes que outras, de modo que em caso de congestionamento, conexões mais importantes tenham maior prioridade.

○ 6.1.2 - Qualidade do serviço

- Resiliência: Probabilidade da camada de transporte, espontaneamente terminar uma conexão devido a problemas internos de congestionamento.
- Esses parâmetros são especificados pelo usuário de transporte quando a conexão é requerida.
- Esse processo é chamado de "*option negotiation*" e uma vez que as opções foram negociadas, elas permanecem do mesmo jeito até o final.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- As primitivas de serviços de transporte OSI englobam tanto serviços orientados à conexão quanto serviços sem conexão.
- Apesar das semelhanças com os serviços prestados pela camada de rede, algumas diferenças são fundamentais. A principal delas é que serviços de rede são desejados para modelar serviços oferecidos por redes reais. Essas redes podem ocasionar perdas de pacotes e problemas do tipo “N-Resets”.
- Já a camada de transporte, do ponto de vista do usuário, ela é livre de erros. Os erros vindos da camada de rede são interceptados pelas entidades de transporte e são corrigidos pelos protocolos de transporte.
- O serviço orientado à conexão fornecido pela camada de transporte possui este objetivo: Esconder as imperfeições da camada de rede de modo que os usuários possam assumir que o fluxo de bits seja totalmente livre de erros.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- Uma configuração normal de conexão faz uso de 4 primitivas (*request, indication, response, confirmation*).
- Exemplo de cenário de conexão:

- T.CONNECT.request: Sinaliza o desejo de estabelecer a conexão com o usuário de transporte ligado ao endereço do ponto de acesso de serviço de transporte (TSAP) nomeado na primitiva T.CONNECT.request.
- Tem-se então uma primitiva T.CONNECT.indication que ocorre no destino.
- o usuário de transporte ligado ao endereço do TSAP recebe a indicação e pode aceitá-la com um T.CONNECT.response ou rejeitá-la com um T.DISCONNECT.request.
- O resultado de uma aceitação chega ao iniciador como um T.CONNECT.confirmation e de uma rejeição, como um T.DISCONNECT.indication.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- No caso do cenário em que o próprio provedor de serviço de transporte rejeita a conexão, nada é transmitido pela rede e então, o lado remoto nem ao menos fica sabendo da falha na tentativa.
- Existem 3 formas em que uma conexão pode ser liberada:

- A forma tradicional é quando uma das partes fazem uso da primitiva do tipo T.DISCONNECT.request que é sinalizada para a outra parte como uma T.DISCONNECT.indication. Então, qualquer uma das partes podem iniciar a liberação da conexão.
- Quando ambas as partes simultaneamente fazem uso da T.DISCONNECT.request, a conexão é liberada sem necessidade de indicação.
- O provedor de transporte pode encerrar a conexão fazendo uso de T.DISCONNECT.indication nas duas extremidades.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- Existem alguns cenários em que o provedor de serviço de transporte irá usar da primitiva T.DISCONNECT.indication espontaneamente.
- Se a rede entra em colapso e se recusa a reagir à repetidas investidas de comunicação, o provedor de serviço de transporte pode usar de tal primitiva para restabelecer a ordem no ambiente de comunicação.
- Nesse sentido, tem-se a quebra da conexão atual devido ao colapso da rede e uma nova conexão pode ser estabelecida.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- O ponto final da conexão possui 4 estados possíveis em um TSAP

- *IDLE*: Nenhuma conexão foi estabelecida ou está tentando ser estabelecida. Ambos os sentidos de conexão são possíveis.
- *Outgoing Connection Pending*: Uma primitiva T.DISCONNECT.request foi realizada, contudo, a resposta do par remoto não foi obtida ainda.
- *Incoming Connection Pending*: Uma primitiva T.DISCONNECT.indication foi obtida, contudo, ainda não foi aceita ou rejeitada.
- *Connection Established*: Uma conexão válida foi estabelecida com sucesso e a parte de troca de dados já pode se iniciar.

○ 6.1.3 - As primitivas de serviços de transporte OSI

- Existem dois caminho para se chegar ao estado de "*Connection Established*" partindo do estado *IDLE*:

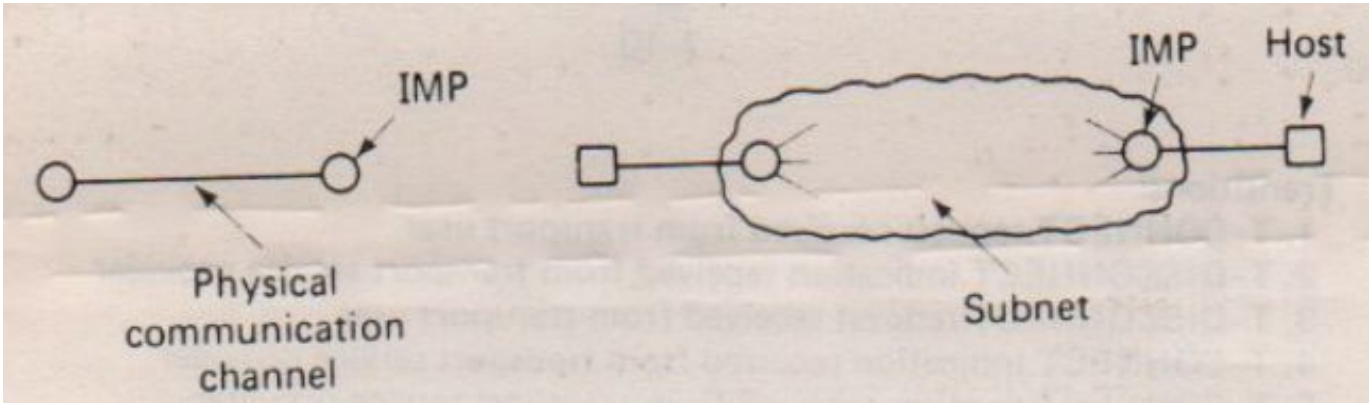
- Caminho pela esquerda: Usado quando a chamada de saída está sendo alocada. O estado intermediário é introduzido depois de uma primitiva T.CONNECT.request ser emitida e espera até que uma primitiva T.CONNECT.confirm apareça.
- Caminho pela direita: Usado quando uma chamada é recebida. O estado intermediário é introduzido depois de uma primitiva T.CONNECT.indication aparecer. Se a conexão for intencionalmente rejeitada ou uma primitiva T.DISCONNECT.indication aparecer, retornamos para o estado IDLE. Caso a conexão seja aceita, o estado "*Connection Established"* é alcançado.

○ 6.1.4 - Protocolos de transporte

- O serviço de transporte é implementado por meio de um protocolo de transporte, utilizado entre as duas entidades de transporte.
- Os protocolos precisam lidar com controle de erro, sequenciamento, controle de fluxo e outras questões.
- São parecidos com protocolos da camada de enlace, porém, possuem diferenças importantes como o fato de que na camada de enlace, dois processadores de mensagem de interface (IMP) se comunicam diretamente por meio de um canal físico. Já na camada de transporte, esse canal físico é substituído pela sub rede inteira.
- Isso gera implicações como o fato de que na camada de enlace, um IMP não precisa especificar com qual IMP o primeiro deseja se comunicar. No caso da camada de transporte, é necessário um endereçamento de destinos explícito para que a comunicação possa ocorrer.

○ 6.1.4 - Protocolos de transporte

- A figura abaixo ilustra a diferença existente no ambiente de comunicação entre entidades IMP. Percebe-se que ao lado esquerdo, tem-se a representação do ambiente na camada de enlace e no lado direito da imagem, o ambiente na camada de transporte.



○ 6.1.4 - Protocolos de transporte

- Outra grande diferença entre a camada de enlace e a de transporte é a existência potencial de capacidade de armazenamento na sub rede. Quando um IMP envia um "*frame"*, ele pode chegar ou ser perdido e não pode ficar "circulando" por um determinado período de tempo e então surgir de forma repentina.
- Na camada de transporte, o grande número de conexões que precisam ser gerenciadas, criam a ideia de dedicar vários *buffers* para cada conexão menos atrativa.
- Para o estudo de protocolos de transporte, tem-se 3 tipos de serviços de rede:

- Tipo A: Todos os serviços fornecidos são perfeitos e os erros como perda de pacotes e duplicação são tão raros que podem ser ignorados.
- Tipo B: Pacotes são raramente perdidos mas de tempos em tempos pode-se ter problemas devido a congestionamento interno, problemas de *hardware* ou *bugs* de *software*. É função do protocolo de transporte reorganizar o ambiente e estabilizar uma nova conexão de rede de modo que os problemas fiquem invisíveis ao usuário de transporte.

○ 6.1.4 - Protocolos de transporte

- A maioria das redes públicas X.25 são do tipo B e os protocolos necessários para o tipo B são mais complexos que os necessários para o tipo A.
- Tipo C: Representa os serviços de rede que não são confiáveis o suficiente. Os protocolos para o tipo C são os mais complexos de todos e precisam lidar com vários tipos de problemas.
- Ou seja, quanto pior o serviço de rede, mais complexo será o protocolo de transporte.
- O modelo OSI idealizou o protocolo de transporte com 5 variáveis (classes).
- Classe 0: É a classe mais simples. Ela configura uma conexão de rede para cada conexão de transporte requerida e assume que a conexão de rede não comete erros. O protocolo de transporte não realiza nenhum tipo de sequenciamento ou controle de fluxo.

○ 6.1.4 - Protocolos de transporte

- Classe 1: Parecido com a Classe 0, exceto pelo fato de que ela foi idealizada para que fosse capaz de se recuperar de "*N-RESETs*". Se uma conexão de rede usada para uma dada conexão de transporte fica sujeita a um *N-RESET*, as duas entidades de transporte são sincronizadas novamente e então podem continuar de onde pararam. A Classe 1 não fornece nenhum tipo de controle de erro ou fluxo.
- Classe 2: Desenvolvido para ser usado com redes confiáveis (Tipo A). Se difere da Classe 0 pois duas ou mais conexões de transporte precisam ser enviadas (multiplexadas) na mesma conexão de rede.
- Classe 3: Esta classe permite multiplexação e também é capaz de se recuperar de *N-RESETs* ao passo que faz uso de controle de fluxo explícito.

○ 6.1.4 - Protocolos de transporte

- Classe 4: Foi desenvolvido para serviços de rede do Tipo C. É capaz de lidar com perdas, duplicações, *N-RESETs*, entre outros. Os protocolos da Classe 4 são os mais complexos de todos
- A escolha de qual classe de protocolo será usada em uma conexão é determinada pelas entidades de transporte no momento em que a conexão é estabelecida. O iniciador propõe uma classe preferida e outras possíveis alternativas enquanto que o outro lado escolhe uma classe de protocolo da lista fornecida. Se nenhuma das opções forem aceitas, a conexão é rejeitada.
- Essa negociação é necessária pois diferentes usuários podem possuir diferentes visões do que possa ser considerado “confiável”.

○ 6.1.5 - Elementos de protocolos de transporte

- As características exatas fornecidas por um protocolo de transporte depende do ambiente em que ele está operando e do tipo de serviço que será oferecido. Contudo, é possível listar elementos básicos que são comuns para vários protocolos de transporte e ainda suas relações com as classe do modelo OSI. Essa lista inclui elementos como “estabelecimento de conexão” , “recusa de conexão”, “transferência TPDU (unidade de dado de protocolo de transporte)”, entre outros.
- Para realizar o movimento de bits pela rede, as entidades de transporte normalmente estabelecem uma conexão de rede. Entretanto, é possível o uso de um protocolo de rede sem conexão para o transporte de dados desde que o protocolo de transporte seja um Classe 4.

○ 6.1.5 - Elementos de protocolos de transporte

- Uma das tarefas dos protocolos de transporte é a de dividir longas mensagens em partes com tamanhos compatíveis com TPDU usados pelos protocolos e então remontar essas partes na outra extremidade.
- Em um cenário de várias conexões, as entidades de transporte darão um número a cada conexão e o colocarão em cada TPDU, de modo que quando este alcançar a outra extremidade, o receptor possa saber qual conexão está associada a ele.
- Portanto, o transporte de TPDUs é uma característica de todos os protocolos de transporte.
- Todos os protocolos de transporte também devem estar aptos a lidar com erros de protocolo de modo a ignorar TPDUs inválidos ou terminar conexões.

○ 6.1.5 - Elementos de protocolos de transporte

- Alguns elementos da lista não se aplicam para todas as classes de protocolos. Um exemplo é o elemento “concatenação de TPDUs”, que é aplicado às classes 1 a 4 mas não à classe 0. Sua função é de permitir que entidades de transporte colete vários TPDUs e envie-os juntos como um único pacote. Isso reduz o número de chamadas da camada de rede.
- Abaixo seguem alguns elementos e sua funções.

- “Controle de fluxo da camada de transporte”: Consiste em se ter uma parte explícita do protocolo de transporte lidando com a questão de quantos TPDUs devem ser enviados a cada instante. Um esquema de janela deslizante pode ser usado.
- “Ressincronização depois de N-RESET”: Permite a cada extremidade saber qual dos TPDUs enviados chegaram ao destino. Existe nas classes 1, 3 e 4.

○ 6.1.5 - Elementos de protocolos de transporte

- “Retransmissão após tempo limite”: Só é necessário para Classe 4 pois é a única classe em que a perda de pacotes é comum o suficiente para necessitar de controle de erro na camada de transporte. A figura abaixo demonstra parte da lista de elementos de protocolo e as classes em que estão presentes.

Protocol element	0	1	2	3	4
Connection establishment	x	x	x	x	x
Connection refusal	x	x	x	x	x
Assignment to network connection	x	x	x	x	x
Splitting long messages into TPDUs	x	x	x	x	x
Association of TPDUs with connection	x	x	x	x	x
TPDU transfer	x	x	x	x	x
Normal release	x	x	x	x	x
Treatment of protocol errors	x	x	x	x	x
Concatenation of TPDUs to the user		x	x	x	x
Error release	x		x		

6.2 - Gerenciamento de conexão

- De vários modos, os protocolos de transporte se assemelham com os protocolos da camada de enlace. Um exemplo disso são os protocolos de janelas deslizantes que podem ser usados nas duas camadas (transporte e enlace).
- Uma diferença fundamental está em como as conexões são gerenciadas pois na camada de enlace, tem-se um pequeno gerenciamento de conexão em que as linhas entre os IMPs estão sempre prontas para uso. Já na camada de transporte, o estabelecimento, a liberação e o gerenciamento das conexões são bem mais complexos. Nesta parte do trabalho, será discutido algumas questões que envolvem o gerenciamento de conexões na camada de transporte bem como os protocolos especiais que são utilizados.

○ 6.2.1 - Endereçamento

- Quando um usuário de transporte quer estabelecer uma conexão com outro, ele deve especificar o usuário remoto para se conectar. O método geralmente usado é o de definir acessos de pontos de serviços de transporte (TSAPs) a quais processos pode ser ligar e esperar pelas requisições de conexões chegarem.
- Conexões de transporte vão de TSAP para TSAP enquanto conexões de rede vão de NSAP para NSAP (acessos de pontos de serviços de rede).
- Se existe muitos processos de servidor que são raramente usados, não é interessante ter todos eles ativos e escutando um endereço TSAP.
- Uma outra abordagem é conhecida como “Protocolo de conexão inicial” da ARPANET. Aqui, cada máquina que deseja oferecer serviço a um usuário remoto possui um processo de servidor especial pelo qual todos os serviços devem ser requeridos.

○ 6.2.1 - Endereçamento

- Essa abordagem da ARPANET funciona bem para servidores que podem ser criados quando necessário, porém existem várias situações em que serviços não existem independente do servidor do processo como por exemplo um servidor de arquivo.
- Para lidar com essa situação, tem-se uma opção alternativa em que existem processos especiais chamados de “servidor nome” ou “servidor diretório”. Para encontrar o endereço TSAP correspondente ao servidor nome, é feito uma conexão do usuário com o servidor nome. O usuário envia uma mensagem especificando o nome do serviço e o servidor nome envia o endereço TSAP. Finalmente, o usuário libera sua conexão com o servidor nome e estabelece outra com o serviço de desejo.

○ 6.2.1 - Endereçamento

- Outra questão interessante seria: Como a entidade de transporte sabe em qual máquina o TSAP está localizado? Isso depende da estrutura do endereço TSAP. Uma possível estrutura são as hierarquias de endereços em que o endereço é formado por sequências de campos usados disjuntos que forma o endereço final.
- Exemplo de endereço TSAP universal: <galáxia,estrela,planeta,país,rede,host,porta>
- Uma alternativa à forma hierárquica, é a chamada “espaço de endereço plano”, em que um segundo nível de mapeamento é necessário para localizar a máquina corretamente.

○ 6.2.2 - Estabelecendo uma conexão

- A princípio, para estabelecer uma conexão é preciso que uma entidade de transporte apenas envie um "*Connection request*" para o destino e espere por uma resposta do tipo "*Connection confirm*". Porém, podem acontecer erros pois a rede perder ou duplicar pacotes. Um ponto crucial que deve ser atenuado é a existência de duplicatas atrasadas de pacotes.
- Ao invés de permitir com que os pacotes fiquem para sempre dentro da subrede, é preciso desenvolver um mecanismo de "matar" pacotes já muito velhos que ainda estão pela rede.
- O tempo de vida de um pacote pode ser restrito a um valor máximo seguindo algumas das seguintes técnicas:

○ 6.2.2 - Estabelecendo uma conexão

- Design restrito de subrede: Inclui qualquer método que previne que pacotes entrem em *looping*, combinado com algum tipo de atraso de congestionamento sobre o maior caminho possível.
- Contador de saltos em cada pacote: Ter um contador de saltos incrementado toda vez que um pacote for encaminhado.
- Carimbo de tempo: Requer que cada pacote possua o tempo em que foi criado e o IMP concorda com o descarte de um pacote mais velho que um determinado tempo acordado previamente.

● Com o tempo de vida dos pacotes limitado, é possível desenvolver uma forma infalível de estabelecer conexões de maneira segura. O método foi descrito por Tomlinson e a idéia básica é a de garantir que dois TPDUs identicamente numerados, nunca são excelentes ao mesmo tempo.

○ 6.2.2 - Estabelecendo uma conexão

- Quando ambas as entidades de transporte concordaram com a sequência de números iniciais, qualquer protocolo de janela deslizante pode ser usado para fluxo de dados.
- O problema acontece quando um host colapsa. Quando ele volta ao normal, a entidade de transporte não sabe onde ele estava anteriormente. Uma solução é exigir que as entidades de transporte fiquem ociosas por um período de tempo, contudo, em uma rede complexa, esse período de tempo pode ser grande e esta estratégia se torna ineficiente.
- O método baseado em relógio soluciona o problema de duplicação atrasada mas para que este método seja útil, é preciso que a conexão seja estabelecida primeiro. Tem-se também o problema de se ter ambos os lados concordando em uma sequência de números iniciais. Para resolver este problema, temos um conceito chamado de "*Three-way Handshake*".

○ 6.2.2 - Estabelecendo uma conexão

- O *Three-way Handshake* é um protocolo de estabelecimento usado em conexões no TCP que não requer que ambos os lados comecem a enviar com a mesma sequência de números, então ele pode ser usado com métodos de sincronização ao invés do método do relógio global.

○ 6.2.3 - Liberando a conexão

- Liberar uma conexão é bem mais fácil que estabelecer uma. Tem-se 3 formas principais que uma conexão pode ser liberada;
- A forma mais usual é por meio de uma primitiva T.DISCONNECT.request realizada por um dos usuários. Após isso, a camada de transporte gera uma primitiva T.DISCONNECT.indication na outra extremidade e a conexão é liberada.
- Outra forma seria quando ambos os usuários fazem uso da primitiva T.DISCONNECT.request simultaneamente.
- A última maneira é quando a camada de transporte realiza uma primitiva T.DISCONNECT.indication nas duas extremidades da conexão.
- Todas essas maneiras de liberar conexões possuem algo em comum: Elas são abruptas e pode resultar em perda de dados.

○ 6.2.3 - Liberando a conexão

- Um protocolo mais sofisticado é necessário para se evitar perda de dados. Um jeito óbvio de contornar este problema é não permitir que algum dos lado realize uma T.DISCONNECT.request até que se tenha certeza que o outro lado recebeu todos os dados que foram enviados. Infelizmente esse tipo de abordagem nem sempre funciona e existe um problema notório chamado de "*Two-army Problem*" que basicamente diz o seguinte: "Se nenhum dos lados está preparado para desconectar até que esteja convencido de que o outro lado está preparado para desconectar também, a desconexão nunca irá acontecer".
- Na prática, um dos lados está disposto a correr o risco ao liberar a conexão portanto a situação não é totalmente devastadora.

○ 6.2.3 - Liberando a conexão

- Se a primitiva T.DISCONNECT.request e suas retransmissões todas forem perdidas, o remetente irá desistir e deletar a conexão enquanto o outro lado nada sabe sobre a tentativa de liberar a conexão. Isso resulta em uma conexão meio-aberta.
- Uma forma de contornar conexões meios-abertas é por meio de uma regra dizendo que se nenhum TPDU chegou em um dado número de segundos, a conexão será automaticamente desconectada. Assim, quando um lado se desconecta, o outro irá deletar a falta de atividade e então se desconectar também. Com essa regra introduzida, é necessário que cada entidade de transporte tenha um contador que irá parar e recomeçar sempre que um TPDU é enviado. Se esse contador expirar, um TPDU fictício é enviado apenas para que o outro lado não se desconecte. Caso muitos TPDUs fictícios são perdidos em uma conexão ociosa, o outro lado irá desconectar automaticamente.
- Portanto, é possível perceber que liberar uma conexão não é uma tarefa tão simples.

○ 6.2.4 - Gerenciamento da conexão baseado em contador

- Recapitulando, a questão original que foi proposta era a de evitar o problema de pacotes antigos, duplicados e T.DISCONNECT.request aparecendo de repente sendo aceitos como legítimos. Uma forma de se lidar com isso é tendo cada entidade de transporte especificando um único identificador de conexão para cada uma, pois assim, é possível reconhecer TPDUs de conexões antigas. Outra forma é usando o já discutido método "*Three-way Handshake*" para estabelecer conexões.
- Tem-se outro método baseado em *timers* (cronômetros, contadores) proposto por Fletcher e Watson. A proposta é a seguinte: Providenciar para que a entidade de transporte se abstenha de deletar informações sobre uma conexão até que todos os TPDUs relacionados a ela estiverem "mortos". Portanto, a entrada da tabela sobre uma conexão de uma entidade de transporte não é deletada quando a conexão é liberada, mas sim quando um intervalo de tempo cuidadosamente escolhido expirar.

○ 6.2.4 - Gerenciamento da conexão baseado em contador

- O esquema funciona basicamente da seguinte maneira: Quando um remetente deseja enviar um fluxo de TPDUs consecutivos, ele cria um registro de conexão internamente que mantém o controle de quais TPDUs foram enviados, quais “acks” foram recebidos, entre outros.
- Sempre que um registro de conexão é criado, um contador é iniciado e sempre que um TPDU é enviado, é enviado usando um registro de conexão criado previamente e o contador é iniciado do começo novamente.
- Se nada for enviado em um certo intervalo de tempo, o contador irá expirar e o registro de conexão é deletado.
- O primeiro TPDU no fluxo contém uma bandeira de 1 bit chamada de “*Data Run Flag*” (DRF) que identifica este TPDU como o primeiro dos TPDUs.

○ 6.2.4 - Gerenciamento da conexão baseado em contador

- Quando algum TPDU é enviado, o contador é iniciado e caso ele for um *ack*, o contador é interrompido. Se o contador desligar, o TPDU é retransmitido, porém, se depois de várias tentativas de retransmissão não for adquirido um *ack*, o remetente irá desistir.
- Quando o remetente recebe o *ack* do último TPDU enviado e localiza a bandeira ARF (A*cknowledgment Run Flag*), todos os contadores de retransmissão são interrompidos.
- Caso o fluxo de TPDUs seja corretamente enviado e recebido, porém alguns *acks* são perdidos, as retransmissões subsequentes irão se perder e o registro de conexão irá ser desligado e deletado.
- Este protocolo mistura propriedades de conexão orientada e sem conexão pois a troca mínima de 2 TPDUs e quando uma sequência de TPDUs é enviada, é garantido que será em ordem. Finalmente, as conexões são liberadas automaticamente graças ao uso dos contadores.